

Universidad Torcuato Di Tella

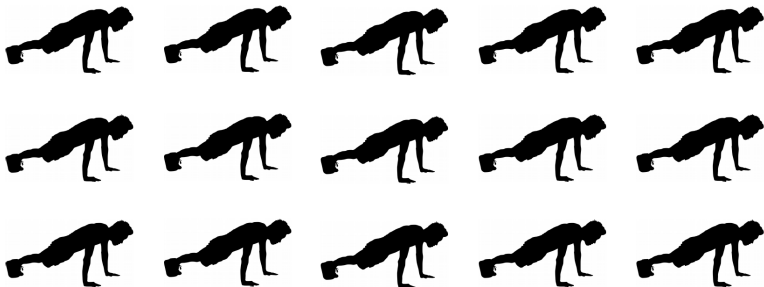
Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Clase teórica 3: Ciclos y funciones

Ciclos

Hacer lo siguiente:



VS.

Repetir 15 veces:



Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

```
while <expr>:  
    <sentencias>
```

- ▶ <expr> es una expresión booleana, llamada *guarda*
- ▶ <sentencias> es el *cuerpo del ciclo*

Mientras la evaluación de la guarda sea **verdadera**, el cuerpo del ciclo se ejecuta una y otra vez. Cuando sea **falsa**, la ejecución continúa con la instrucción inmediatamente posterior al cuerpo del ciclo.

Sintaxis de Python:¹

- ▶ A continuación de la guarda se escribe el carácter dos puntos ':'
- ▶ Los bloques de código están delimitados por tabulación

¹ Estas reglas son equivalentes a las de las sentencias condicionales y, más en general, a toda la organización de código en Python.

Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

Ejemplo:

Programa

```
⇒ i = 0
   while i<3:
       print(i)
       i = i+1
   ...
```

Memoria

i → ?

Consola



Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

Ejemplo:

Programa

```
⇒ i = 0
   while i<3:
       print(i)
       i = i+1
   ...
```

Memoria

i → 0

Consola



Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

Ejemplo:

Programa

```
    i = 0
    while i<3:
⇒      print(i)
        i = i+1
    ...
```

Memoria

```
i → 0
```

Consola



Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

Ejemplo:

Programa

```
i = 0
while i<3:
    print(i)
    i = i+1
⇒ ...
```

Memoria

i → 0

Consola

0

Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

Ejemplo:

Programa

```
⇒ i = 0
   while i<3:
       print(i)
       i = i+1
   ...
```

Memoria

i → 1

Consola

0

Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

Ejemplo:

Programa

```
    i = 0
    while i<3:
⇒      print(i)
        i = i+1
    ...
```

Memoria

i → 1

Consola

0

Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

Ejemplo:

Programa

```
i = 0
while i<3:
    print(i)
    i = i+1
⇒ ...
```

Memoria

i → 1

Consola

```
0
1
```

Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

Ejemplo:

Programa

```
⇒ i = 0
   while i<3:
       print(i)
       i = i+1
   ...
```

Memoria

i → 2

Consola

```
0
1
```

Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

Ejemplo:

Programa

```
    i = 0
    while i<3:
⇒      print(i)
        i = i+1
    ...
```

Memoria

i → 2

Consola

```
0
1
```

Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

Ejemplo:

Programa

```
i = 0
while i<3:
    print(i)
    i = i+1
⇒ ...
```

Memoria

i → 2

Consola

```
0
1
2
```

Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

Ejemplo:

Programa

```
⇒ i = 0
   while i<3:
       print(i)
       i = i+1
   ...
```

Memoria

i → 3

Consola

```
0
1
2
```

Ciclos

Los **ciclos** permiten que un programa repita una serie de instrucciones mientras una condición sea verdadera.

Ejemplo:

Programa

```
i = 0
while i<3:
    print(i)
    i = i+1
⇒ ...
```

Memoria

i → 3

Consola

```
0
1
2
```

Ciclos

Cuando se necesita recorrer una secuencia de valores numéricos consecutivos, es habitual utilizar la función `range(n)`, que genera dicha secuencia de manera automática.

A continuación se muestran dos programas equivalentes: uno utiliza un ciclo `while` con un contador explícito, y el otro emplea un ciclo `for` con `range`.

Programa 1

```
i = 0
while i<3:
    print(i)
    i = i+1
...
```

Programa 2

```
for i in range(3):
    print(i)
...
```

La función `range` puede invocarse con dos parámetros para definir el inicio y el fin de la secuencia. Por ejemplo, `range(3, 7)` genera los valores 3, 4, 5 y 6.

Ejercicios en Python

Recomendación: resolver cada ejercicio en un archivo diferente (ej: ej1.py, ej2.py, etc.).

Ejercicio. (1) Observar la diferencia que se produce al ejecutar los siguientes dos programas en Python:

```
print(1)
print(2)
print(3)
```

```
print(1, end=' ')
print(2, end=' ')
print(3)
```

Ejercicio. (2) Escribir un programa en Python que, usando un ciclo, imprima por pantalla los números del 1 al 5 en una única línea.

Ejercicio. (3) Escribir un programa en Python que, usando ciclos, imprima por pantalla lo que muestra la imagen:

```
Ana: ¡Yo tengo escalera!
1 2 3 4 5
Agustín: ¡Yo también!
1 2 3 4 5
```

Funciones

De la Sierra Morena,
cielito lindo, vienen bajando
un par de ojitos negros,
cielito lindo, de contrabando.

Siempre que te enamores
mira primero, mira primero
dónde pones los ojos,
cielito lindo, no llores luego.

¡Ay, ay, ay, ay!
canta y no llores,
porque cantando se alegran,
cielito lindo, los corazones.

¡Ay, ay, ay, ay!
canta y no llores,
porque cantando se alegran,
cielito lindo, los corazones.

Ese lunar que tienes,
cielito lindo, junto a la boca
no se lo des a nadie,
cielito lindo, que a mí me toca.

De tu reja a la mía,
cielito lindo, no hay más que un paso
ora que estamos solos,
cielito lindo, dame un abrazo.

¡Ay, ay, ay, ay!
canta y no llores,
porque cantando se alegran,
cielito lindo, los corazones.

¡Ay, ay, ay, ay!
canta y no llores,
porque cantando se alegran,
cielito lindo, los corazones.

Funciones

De la Sierra Morena,
cielito lindo, vienen bajando
un par de ojitos negros,
cielito lindo, de contrabando.

[Estribillo]

Ese lunar que tienes,
cielito lindo, junto a la boca
no se lo des a nadie,
cielito lindo, que a mí me toca.

[Estribillo]

Siempre que te enamores
mira primero, mira primero
dónde pones los ojos,
cielito lindo, no llores luego.

[Estribillo]

De tu reja a la mía,
cielito lindo, no hay más que un paso
ora que estamos solos,
cielito lindo, dame un abrazo.

[Estribillo]

Estribillo

¡Ay, ay, ay, ay!
canta y no llores,
porque cantando se alegran,
cielito lindo, los corazones.

Funciones

Algo parecido puede ocurrir en los programas...

```
print('Ana: ¡Yo tengo escalera!')
for i in range (1,6):
    print(i, end='')
print()
print('Agustín: ¡Yo también!')
for i in range (1,6):
    print(i, end='')
print()
print('Pablo: ¡Mirá qué casualidad!')
for i in range (1,6):
    print(i, end='')
print()
print('Julieta: ¿Esto es un empate?')
for i in range (1,6):
    print(i, end='')
print()
```

Funciones

Esta solución es cualitativamente mejor:

```
print('Ana: ¡Yo tengo escalera!')
print(obtener_escalera())
print('Agustín: ¡Yo también!')
print(obtener_escalera())
print('Pablo: ¡Mirá qué casualidad!')
print(obtener_escalera())
print('Julietta: ¿Esto es un empate?')
print(obtener_escalera())
```

Funciones

Una **función** es una porción de un programa más grande que encapsula una tarea específica. Una vez definida, puede invocarse desde cualquier punto del programa igual que cualquier otra instrucción.

- ▶ Permite **dividir un problema** en subproblemas más simples
- ▶ Permite **ordenar conceptualmente** el código para que sea más fácil de entender
- ▶ Permite **reutilizar soluciones** a problemas pequeños en la solución de problemas mayores

Funciones

Ejemplo: definición de una función que convierte millas a kilómetros.

```
def millas_a_kilometros():  
    mi = 50  
    km = mi * 1.60934  
    return km  
  
print(50, ' mi = ', millas_a_kilometros(), ' km')
```

- ▶ La palabra **def** indica que estamos definiendo una función
- ▶ A continuación se escribe el **nombre de la función** (¡usar nombres descriptivos!), seguido dos puntos ':'
- ▶ Los paréntesis se usan para definir **parámetros** (ya lo veremos)
- ▶ La función devuelve el resultado de evaluar la expresión a continuación de **return**
- ▶ En Python las funciones tienen que declararse antes de ser invocadas (más arriba)

Funciones

Usar funciones facilita la [localización y corrección de errores](#).

```
# Programa 1
mi = 50
km = mi + 1.60934      X
print(50, ' mi = ', km, ' km')
...
mi = 50
km = mi + 1.60934      X
print(50, ' mi = ', km, ' km')
```

```
# Programa 2
def millas_a_kilometros():
    mi = 50
    km = mi + 1.60934    X
    return km

print(50, ' mi = ', millas_a_kilometros(), ' km')
...
print(50, ' mi = ', millas_a_kilometros(), ' km')
```


Funciones

Ejercicio. (4) Eliminar la redundancia en el siguiente programa:

```
print('Ana: ¡Yo tengo escalera!')
for i in range (1,6)
    print(i, end=' ')
print()
print('Agustin: ¡Yo tambien!')
for i in range (1,6)
    print(i, end=' ')
print()
print('Pablo: ¡Mira que casualidad!')
for i in range (1,6)
    print(i, end=' ')
print()
```

```
Ana: ¡Yo tengo escalera!
1 2 3 4 5

Agustín: ¡Yo también!
1 2 3 4 5

Pablo: ¡Mirá qué casualidad!
1 2 3 4 5
```

Parámetros

Un elefante se balanceaba
sobre la tela de una araña
como veía que resistía
fue a llamar a otro elefante

Dos elefantes se balanceaban
sobre la tela de una araña
como veían que resistía
fueron a llamar a otro elefante

Tres elefantes se balanceaban
sobre la tela de una araña
como veían que resistía
fueron a llamar a otro elefante

Cuatro elefantes se balanceaban
sobre la tela de una araña
como veían que resistía
fueron a llamar a otro elefante

Cinco elefantes se balanceaban
sobre la tela de una araña
como veían que resistía
fueron a llamar a otro elefante

Seis elefantes se balanceaban
sobre la tela de una araña
como veían que resistía
fueron a llamar a otro elefante

Parámetros

Canción	Estribillo (<i>Cantidad</i>)
Un elefante se balanceaba sobre la tela de una araña como veía que resistía fue a llamar a otro elefante	<i>Cantidad</i> elefantes se balanceaban sobre la tela de una araña como veían que resistía fueron a llamar a otro elefante
[Estribillo <i>Dos</i>] [Estribillo <i>Tres</i>] [Estribillo <i>Cuatro</i>] [Estribillo <i>Cinco</i>] [Estribillo <i>Seis</i>]	

El estribillo tiene un parámetro: el valor que adquiere *Cantidad* se define cada vez que el estribillo es referenciado.

Parámetros

Ejemplo: Función que convierte cualquier cantidad de millas a kilómetros.

```
def millas_a_kilometros(mi):  
    km = mi * 1.60934  
    return km  
  
print(50, ' mi = ', millas_a_kilometros(50), ' km')  
print(63, ' mi = ', millas_a_kilometros(63), ' km')  
print(84, ' mi = ', millas_a_kilometros(84), ' km')  
print(99, ' mi = ', millas_a_kilometros(99), ' km')  
...
```

- ▶ Se puede incluir **cualquier cantidad de parámetros** separados por comas (¡usar nombres descriptivos!)
- ▶ Dentro de la función se **usan** del mismo modo que las variables
- ▶ Se invoca a la función usando **argumentos** (50, 63, 84, 99, ...)
- ▶ Usando parámetros se construyen **soluciones generales** que permiten resolver un gran número de problemas particulares

Parámetros

Ejercicio. (5) Escribir una función `primeros_numeros(n)` en Python que, dado un número `n`, devuelva un `str` con los números de 1 al `n`. A continuación puede observarse un programa que invoca a la función, junto con el resultado esperado.

```
print(primeros_numeros(1))  
print(primeros_numeros(3))  
print(primeros_numeros(6))  
print(primeros_numeros(10))
```

```
1  
1 2 3  
1 2 3 4 5 6  
1 2 3 4 5 6 7 8 9 10
```

Observaciones finales

Cada ejecución de una función tiene su **propio espacio de memoria**, como si fuera otro programa.

```
def raiz_cuadrada(n):  
    i = 1  
    while i * i <= n:  
        i = i + 1  
    return i - 1  
  
x = 1  
while x <= 5:  
    print(raiz_cuadrada(x))  
    x = x + 1
```

Las variables **n** e **i** son alcanzables dentro de `raiz_cuadrada`, pero no desde el cuerpo principal del programa.

Observaciones finales

Cada ejecución de una función tiene su **propio espacio de memoria**, como si fuera otro programa.

```
def raiz_cuadrada(n):  
    i = 1  
    while i * i <= n:  
        i = i + 1  
    return i - 1  
  
def potencia(a, n):  
    rv = 1  
    i = 0  
    while i < n:  
        rv = rv * a  
        i = i + 1  
    return rv
```

$i \neq i$ y $n \neq n$

Funciones y parámetros

Ejercicio. (tarea)

- (a) Escribir en Python una función `es_bisiesto(año)` que devuelva `True` si el año `año` es bisiesto y `False` en caso contrario.

Nota: un año es bisiesto tanto si es divisible por 400 como si es divisible por 4 pero no por 100.

- (b) Escribir en Python una función `contar_bisiestos(desde, hasta)` que devuelva la cantidad de bisiestos que hay entre el año `desde` y el año `hasta`.

Ayuda: desde `contar_bisiestos(desde, hasta)`, invocar a la función definida en el punto (a).

Repaso de la clase de hoy

- ▶ Ciclos
- ▶ Funciones
 - ▶ Definición
 - ▶ Parámetros
 - ▶ Argumentos
 - ▶ Valor de retorno

Bibliografía complementaria:

- ▶ IPP, capítulo 4 completo, secciones 6.1 a 6.4 y 7.2 a 7.9.

Con lo visto, ya pueden resolver las secciones 3 y 4 de la guía de ejercicios.